

KuraVet — Backlog Sprint 3

Sistema de Cuidado Contínuo Pet

Integrantes

- Pedro Henrique Luiz Alves Duarte – RM563405
- Henrique Martins Oliveira – RM563620
- Guilherme Macedo Martins – RM562396

1. Visão Geral da Sprint 3

A Sprint 3 consolida a fundação técnica multi-camada do KuraVet: modelo de dados core em Oracle, infraestrutura Azure provisionada via CLI, microsserviço .NET de observabilidade, monólito Java com segurança e API para mobile, e o app React Native com navegação, integração de API e autenticação desacoplada via Firebase.

Total de esforço planejado na Sprint 3: 91 pontos, distribuídos em 1 squads. Sprints anteriores (assumidas concluídas): Sprint 1 (setup de ambientes, repositórios, pipelines base) e Sprint 2 (modelagem ER, wireframes de telas, PoCs de conectividade Azure/Firebase).

Escopo deste documento: cobre o backlog planejado do produto Épico, 6 Features e 18 PBIs com o Release Plan (seção 3) mostrando a Sprint que entrega cada item (Sprint 3 e 4). Apenas a **Sprint 3** (em curso) tem as Tasks detalhadas com descrição, esforço e dependências (seção 2), conforme o item "Detalhamento da Sprint atual" pede; os PBIs da Sprint 4 têm Descrição, Critérios de Aceite, Prioridade, Esforço e Dependências definidos, mas ainda sem quebra em Tasks isso só será feito quando essa sprint virar a sprint corrente.

2. Hierarquia do Backlog

2.0 Sequência de Implementação do Backlog Completo

A ordem abaixo respeita simultaneamente Prioridade e Dependências técnicas de cada PBI, cobrindo todo o backlog do produto (Sprint 3, atual, até Sprint 4, mais distante no roadmap). É a sequência que deve guiar o time na hora de decidir "o que puxar primeiro" dentro e entre sprints.

Ordem	PBI	Sprint	Prioridade	Esforço	Depende de
1	PBI 1.1 — Modelagem e Script Core do BD	3	1	8	—
2	PBI 2.1 — Provisionamento via Azure CLI	3	1	8	—
3	PBI 1.2 — Procedures e Functions de Negócio	3	1	13	PBI 1.1
4	PBI 1.3 — Trigger de Auditoria	3	2	5	PBI 1.1

Ordem	PBI	Sprint	Prioridade	Esforço	Depende de
5	PBI 2.2 — CRUD Funcional na Nuvem	3	2	8	PBI 2.1, PBI 1.1
6	PBI 3.1 — Health Check, Logs e Métricas	3	2	8	—
7	PBI 4.1 — Views Thymeleaf e Flyway	3	2	8	PBI 1.1 (ref.)
8	PBI 5.3 — Autenticação Firebase	3	2	5	Projeto Firebase configurado
9	PBI 4.2 — Segurança por Perfis	3	2	5	PBI 4.1
10	PBI 5.1 — Navegação e Telas Protegidas	3	2	8	PBI 5.3
11	PBI 3.2 — Testes Automatizados .NET	3	3	5	PBI 3.1
12	PBI 4.3 — API JSON Paralela para Mobile	3	3	5	PBI 4.2, PBI 1.1
13	PBI 5.2 — Integração de API (TanStack Query)	3	3	5	PBI 4.3, PBI 5.1
14	PBI 6.1 — Pipeline CI/CD	4	1	13	Todos os PBIs da Sprint 3
15	PBI 6.4 — Observabilidade Centralizada	4	2	8	PBI 3.1, PBI 6.1
16	PBI 6.3 — Hardening de Compliance Cross-Sistema	4	2	8	PBI 1.3
17	PBI 6.2 — Testes E2E Integrados	4	2	13	PBI 6.1, Mobile + API em produção
18	PBI 6.5 — Notificações Push (Vacina/Consulta)	4	3	13	PBI 5.3, PBI 4.3

ÉPICO — KuraVet: Plataforma de Cuidado Contínuo Pet

Descrição: Plataforma multi-camada para gestão do cuidado contínuo de pets (prontuário, vacinação, consultas e planos de acompanhamento), com backend de dados robusto em Oracle, infraestrutura em nuvem, serviços de backend distribuídos (observabilidade em .NET e regras de negócio administrativas em Java) e aplicativo mobile para tutores.

História de Usuário (guarda-chuva): Como tutor de um pet, quero uma plataforma completa de cuidado contínuo, para que eu acompanhe vacinas, consultas e o histórico de saúde do meu pet em um só lugar com a clínica conseguindo gerenciar tudo de forma segura, tanto pelo sistema administrativo quanto pelo app.

FEATURE 1 — Camada de Dados (Oracle)

Núcleo relacional do domínio KuraVet, incluindo lógica de negócio em PL/SQL e auditoria. *História de Usuário:* Como plataforma, preciso de uma base de dados robusta e auditável, para que todas as demais funcionalidades (mobile, web, APIs) tenham informação confiável e rastreável.

PBI 1.1 — Modelagem e Script Core do Banco de Dados

- **Descrição:** Criar `script_bd.sql` contendo o modelo relacional core do KuraVet (tutores, pets, espécies/raças, prontuários, consultas, vacinas e planos de cuidado contínuo), com constraints (PK/FK/CHECK), sequences e massa de dados de exemplo.
- **História de Usuário:** Como administrador do sistema, quero um modelo de dados core confiável e populado com massa de teste, para que as demais camadas (backend, mobile) tenham uma base de dados consistente para desenvolver e testar sobre ela.
- **Critérios de Aceite:**
 - Script cria todas as tabelas core sem erros em um schema Oracle limpo.
 - Cada tabela contém exatamente **5 registros** de massa de teste, com integridade referencial válida.
 - Nomenclatura de objetos segue padrão definido (`tb_`, `pk_`, `fk_`, `ck_`).
 - Script é idempotente (trata reexecução sem falhar, via `DROP IF EXISTS`/checagem de existência).
- **Prioridade:** 1 (Mais alta)
- **Esforço:** 8 pontos
- **Dependências técnicas:** Modelo ER aprovado (Sprint 2); ambiente Oracle disponível (Squad Cloud).

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T1.1.1	Modelagem lógica e DDL das tabelas core	3	—
T1.1.2	Constraints (PK/FK/CHECK) e sequences	2	T1.1.1
T1.1.3	INSERTs com 5 registros por tabela (massa consistente)	2	T1.1.2
T1.1.4	Validação de execução idempotente do <code>script_bd.sql</code>	1	T1.1.3

PBI 1.2 — Procedures e Functions de Negócio (PL/SQL)

- **Descrição:** Implementar lógica server-side manual: 2 procedures (uma com JOIN retornando JSON; outra calculando subtotais sem `ROLLUP/CUBE`) e 2 functions (uma com montagem manual de JSON proibido usar conversão JSON nativa do Oracle; outra implementando uma regra de negócio de status de cuidado contínuo).

- **História de Usuário:** Como veterinário responsável pelo cuidado do pet, quero que o sistema calcule automaticamente o status do plano de cuidado e me devolva o histórico do pet em formato estruturado, para que eu tome decisões rápidas sobre consultas e vacinas pendentes.
- **Critérios de Aceite:**
 - **PRC_PET_PRONTUARIO_JSON:** JOIN entre pet, tutor, consultas e vacinas, retornando JSON válido.
 - **PRC_SUBTOTAL_CONSULTAS_POR_PET:** subtotais por pet e total geral **sem ROLLUP/CUBE/GROUPING SETS** (agregação manual via **GROUP BY + UNION ALL**).
 - **FN_TO_JSON_MANUAL:** monta string JSON via concatenação, **sem** usar **JSON_OBJECT/JSON_ARRAY/TO_JSON** ou qualquer função nativa de conversão JSON do Oracle.
 - **FN_CALC_STATUS_CUIDADO:** aplica regra de negócio (janelas de vencimento de vacina/consulta) e retorna status categorizado (**EM_DIA / ATRASADO / CRITICO**).
 - Cada procedure/function trata **no mínimo 3 cláusulas 'EXCEPTION WHEN'** distintas (ex.: **NO_DATA_FOUND, TOO_MANY_ROWS, VALUE_ERROR, OTHERS**).
- **Prioridade:** 1 (Mais alta)
- **Esforço:** 13 pontos
- **Dependências técnicas:** PBI 1.1 concluído (schema + massa de dados).

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T1.2.1	Procedure PRC_PET_PRONTUARIO_JSON (JOIN + JSON)	3	PBI 1.1
T1.2.2	Procedure PRC_SUBTOTAL_CONSULTAS_POR_PET (subtotal manual, sem ROLLUP/CUBE)	3	PBI 1.1
T1.2.3	Function FN_TO_JSON_MANUAL (concatenação, sem função nativa)	3	T1.2.1
T1.2.4	Function FN_CALC_STATUS_CUIDADO (regra de negócio)	3	PBI 1.1
T1.2.5	Tratamento de exceções (3 EXCEPTION WHEN /bloco) e testes manuais	1	T1.2.1–T1.2.4

PBI 1.3 — Trigger de Auditoria

- **Descrição:** Implementar trigger de auditoria em tabela crítica (ex.: prontuário), capturando alterações via **:OLD** e **:NEW** e persistindo em tabela de log de auditoria — suporte a compliance/LGPD.
- **História de Usuário:** Como responsável por compliance do KuraVet, quero que toda alteração em dados sensíveis do prontuário seja auditada automaticamente, para que eu consiga comprovar rastreabilidade em caso de fiscalização ou disputa (LGPD).
- **Critérios de Aceite:**
 - Trigger dispara em **INSERT, UPDATE** e **DELETE**.
 - Registro de auditoria contém valores antigos (**:OLD**), novos (**:NEW**), usuário, timestamp e operação.
 - Tabela de auditoria própria criada.
 - Trigger trata **no mínimo 3 'EXCEPTION WHEN'**.
 - Testes comprovam rastreabilidade completa das alterações.

- **Prioridade:** 2
- **Esforço:** 5 pontos
- **Dependências técnicas:** PBI 1.1.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T1.3.1	Criar tabela tb_auditoria_log	1	PBI 1.1
T1.3.2	Implementar TRG_AUD_PRONTUARIO (INSERT/UPDATE/DELETE, :OLD/:NEW)	2	T1.3.1
T1.3.3	Tratamento de exceções (3 EXCEPTION WHEN) no trigger	1	T1.3.2
T1.3.4	Testes de rastreabilidade e evidência de compliance	1	T1.3.3

FEATURE 2 — Infraestrutura Azure (DevOps/Cloud)

Provisionamento via Azure CLI e primeira carga funcional em nuvem. *História de Usuário:* Como time de operações, preciso de infraestrutura provisionada de forma automatizada e segura, para que o sistema rode de forma confiável e escalável na nuvem.

PBI 2.1 — Provisionamento via Azure CLI

- **Descrição:** Provisionar toda a infraestrutura necessária (Resource Group, ACR, ACI ou serviço PaaS, Key Vault, Log Analytics) **exclusivamente via Azure CLI**, sem uso do portal, com arquitetura 100% containerizada (ACR + ACI sem privilégios root) ou 100% PaaS.
- **História de Usuário:** Como engenheiro de infraestrutura, quero provisionar todos os recursos de nuvem de forma automatizada e reproduzível via linha de comando, para que o ambiente possa ser recriado rapidamente sem depender de configuração manual no portal.
- **Critérios de Aceite:**
 - Todos os recursos criados via comandos **az** versionados em script.
 - Decisão arquitetural (containerizado vs. PaaS) documentada e aplicada de forma consistente (sem mix).
 - Se containerizado: imagem valida execução em contexto **non-root**.
 - Scripts idempotentes e documentados (pré-requisitos, ordem de execução).
 - Segredos não expostos em texto plano (Key Vault ou equivalente).
- **Prioridade:** 1 (Mais alta)
- **Esforço:** 8 pontos
- **Dependências técnicas:** Subscription Azure ativa; decisão arquitetural aprovada.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T2.1.1	Script AZ CLI — Resource Group e ACR	2	—
T2.1.2	Build & push de imagem non-root para o ACR	2	T2.1.1

Task	Descrição	Esforço	Dependências
T2.1.3	Provisionamento de ACI (ou serviço PaaS) via AZ CLI	2	T2.1.2
T2.1.4	Key Vault/segredos e Log Analytics	1	T2.1.1
T2.1.5	Validação non-root e documentação do provisionamento	1	T2.1.3, T2.1.4

PBI 2.2 — CRUD Funcional na Nuvem

- **Descrição:** Implementar CRUD funcional em nuvem para duas tabelas do domínio (ex.: Pet e Consulta), expondo endpoints operacionais no recurso provisionado.
- **História de Usuário:** Como desenvolvedor backend, quero um CRUD funcional já rodando na nuvem, para que eu consiga validar que a camada de dados provisionada realmente sustenta operações reais antes de conectar os demais serviços.
- **Critérios de Aceite:**
 - CRUD completo (Create/Read/Update/Delete) para as 2 tabelas, testável via HTTP real no ambiente Azure.
 - Persistência sobrevive a reinícios (storage/DB gerenciado).
 - Sem privilégios administrativos/root expostos.
- **Prioridade:** 2
- **Esforço:** 8 pontos
- **Dependências técnicas:** PBI 2.1; PBI 1.1 (referência de modelo de dados).

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T2.2.1	Definição dos endpoints CRUD (Pet, Consulta)	1	PBI 2.1
T2.2.2	Implementação do CRUD e conexão com banco cloud	3	T2.2.1
T2.2.3	Deploy do CRUD no recurso provisionado	2	T2.2.2
T2.2.4	Testes end-to-end no ambiente Azure	2	T2.2.3

FEATURE 3 — Microserviço .NET (Observabilidade e Qualidade)

História de Usuário: Como equipe de operações, preciso de visibilidade sobre a saúde dos serviços, para que problemas sejam identificados antes de afetar os usuários finais.

PBI 3.1 — Health Check, Logs Estruturados e Métricas

- **Descrição:** Criar microserviço .NET isolado, focado em observabilidade, com endpoints de health check, logging estruturado via Serilog e métricas via OpenTelemetry.

- **História de Usuário:** Como engenheiro de operações (SRE), quero que o microserviço exponha health checks, logs estruturados e métricas, para que eu consiga monitorar a saúde do sistema em produção e diagnosticar problemas rapidamente.
- **Critérios de Aceite:**
 - Endpoints **/health** (liveness) e **/health/ready** (readiness) implementados.
 - Logs estruturados em JSON via Serilog, com correlação de request id.
 - Métricas expostas via OpenTelemetry (exportador configurado).
 - Serviço isolado, sem acoplamento a outros serviços do KuraVet.
- **Prioridade:** 2
- **Esforço:** 8 pontos
- **Dependências técnicas:** Nenhuma (serviço isolado); Docker disponível.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T3.1.1	Criar projeto .NET isolado e estrutura base	1	—
T3.1.2	Health Checks (liveness/readiness)	2	T3.1.1
T3.1.3	Configurar Serilog (logs estruturados)	2	T3.1.1
T3.1.4	Configurar OpenTelemetry (métricas)	2	T3.1.1
T3.1.5	Containerização do microserviço	1	T3.1.2–T3.1.4

PBI 3.2 — Testes Automatizados (xUnit, Moq, AAA)

- **Descrição:** Escrever testes de unidade e integração do microserviço, usando xUnit, Moq e o padrão AAA.
- **História de Usuário:** Como desenvolvedor .NET, quero testes automatizados de unidade e integração cobrindo o microserviço, para que eu tenha confiança de que mudanças futuras não quebrem a observabilidade do sistema.
- **Critérios de Aceite:**
 - Testes de unidade cobrindo lógica isolada, com Moq para dependências.
 - Testes de integração validando os endpoints de health check (**WebApplicationFactory**).
 - Todos os testes seguem estrutura AAA explícita.
 - Cobertura mínima (ex.: 70%) documentada.
- **Prioridade:** 3
- **Esforço:** 5 pontos
- **Dependências técnicas:** PBI 3.1.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T3.2.1	Testes de unidade (xUnit + Moq, AAA)	2	PBI 3.1

Task	Descrição	Esforço	Dependências
T3.2.2	Testes de integração de endpoints (WebApplicationFactory)	2	PBI 3.1
T3.2.3	Relatório de cobertura e evidências	1	T3.2.1, T3.2.2

FEATURE 4 — Monólito Java (Spring Boot)

História de Usuário: Como equipe administrativa da clínica, preciso de um painel de gestão seguro, para que eu consiga administrar o sistema e alimentar o aplicativo mobile com dados atualizados.

PBI 4.1 — Views Thymeleaf e Banco Versionado (Flyway)

- **Descrição:** Desenvolver módulo administrativo monolítico com views server-side em Thymeleaf e banco versionado via Flyway.
- **História de Usuário:** Como administrador da clínica veterinária, quero telas administrativas simples para cadastrar e listar informações do KuraVet, para que eu consiga gerenciar o sistema sem depender de acesso direto ao banco de dados.
- **Critérios de Aceite:**
 - Telas administrativas (listagem/cadastro) funcionais via Thymeleaf.
 - Migrations Flyway versionadas (**V1__...**, **V2__...**) aplicadas automaticamente no startup.
 - Rollback de migration testado em ambiente de homologação.
- **Prioridade:** 2
- **Esforço:** 8 pontos
- **Dependências técnicas:** PBI 1.1 (referência de modelo, se schema compartilhado).

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T4.1.1	Setup Spring Boot + Flyway (migrations iniciais)	2	—
T4.1.2	Views Thymeleaf (listagem/cadastro)	3	T4.1.1
T4.1.3	Testes de migration (apply/rollback)	2	T4.1.1
T4.1.4	Ajustes de UX e validação server-side de formulário	1	T4.1.2

PBI 4.2 — Segurança por Perfis (Spring Security)

- **Descrição:** Proteger rotas do monólito com Spring Security, definindo dois perfis de acesso (ex.: **ADMIN** e **VETERINARIO**) com autorizações distintas.
- **História de Usuário:** Como gestor da clínica, quero que o sistema restrinja o acesso conforme o perfil do usuário (administrador ou veterinário), para que informações sensíveis só sejam acessadas por quem tem permissão.
- **Critérios de Aceite:**

- Login funcional protegendo rotas administrativas.
- Dois perfis configurados com permissões distintas e testadas (acesso negado/permitido conforme perfil).
- Senhas armazenadas com hashing seguro (BCrypt).
- Testes de autorização por perfil documentados.
- **Prioridade:** 2
- **Esforço:** 5 pontos
- **Dependências técnicas:** PBI 4.1.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T4.2.1	Configuração Spring Security + autenticação	2	PBI 4.1
T4.2.2	Perfis (ADMIN/VETERINARIO) e regras de rota	2	T4.2.1
T4.2.3	Testes de autorização por perfil	1	T4.2.2

PBI 4.3 — API JSON Paralela para o App Mobile

- **Descrição:** Expor API REST paralela (JSON), a partir do mesmo monólito, para consumo pelo app mobile, reaproveitando as regras de negócio internas.
- **História de Usuário:** Como tutor de pet usando o aplicativo mobile, quero que o app consiga buscar meus dados e os do meu pet através de uma API, para que eu tenha acesso às informações de cuidado contínuo diretamente pelo celular.
- **Critérios de Aceite:**
 - Endpoints REST em JSON para Pet, Consulta e Plano de Cuidado.
 - API protegida (token/sessão) compatível com o app mobile.
 - Contrato documentado (OpenAPI/Swagger).
 - Testado com client externo (Postman/Insomnia) simulando o app mobile.
- **Prioridade:** 3
- **Esforço:** 5 pontos
- **Dependências técnicas:** PBI 4.2; PBI 1.1.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T4.3.1	Endpoints REST JSON (Pet, Consulta, Plano)	2	PBI 4.2
T4.3.2	Proteção da API (token/sessão compatível com mobile)	2	T4.3.1
T4.3.3	Documentação OpenAPI/Swagger e testes com client externo	1	T4.3.1

FEATURE 5 — App Mobile (React Native)

História de Usuário: Como tutor de pet, preciso de um aplicativo confiável, seguro e fácil de navegar, para que eu acompanhe o cuidado do meu pet a qualquer momento, de qualquer lugar.

PBI 5.1 — Navegação Real e Telas Protegidas

- **Descrição:** Implementar navegação real (React Navigation) com no mínimo 6 telas protegidas por autenticação (ex.: Login, Home, Pets, Detalhe do Pet, Agenda, Perfil).
- **História de Usuário:** Como tutor de pet, quero navegar entre as principais telas do aplicativo (login, home, pets, agenda, perfil) de forma fluida e protegida por login, para que eu só acesse minhas informações depois de autenticado.
- **Critérios de Aceite:**
 - Mínimo de **6 telas** navegáveis via rotas reais (Stack/Tab Navigator).
 - Telas protegidas redirecionam para login quando o usuário não está autenticado.
 - Fluxo de navegação validado por roteiro de QA.
- **Prioridade:** 2
- **Esforço:** 8 pontos
- **Dependências técnicas:** PBI 5.3 (autenticação), para proteção efetiva das rotas.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T5.1.1	Setup React Navigation (Stack/Tab)	1	—
T5.1.2	Implementação das 6 telas (estrutura e layout)	4	T5.1.1
T5.1.3	Guard de rotas protegidas (redirecionamento para login)	2	PBI 5.3
T5.1.4	Testes manuais de navegação (roteiro de QA)	1	T5.1.2, T5.1.3

PBI 5.2 — Integração de API (TanStack Query)

- **Descrição:** Consumir a API do backend Java via TanStack Query, com estados de loading e timeout estendido de 60 segundos.
- **História de Usuário:** Como tutor de pet, quero que o aplicativo mostre um indicador de carregamento e não trave ao buscar dados da API, mesmo em conexões lentas, para que eu tenha uma experiência confiável mesmo fora de uma boa rede.
- **Critérios de Aceite:**
 - Requisições usam TanStack Query (**useQuery/useMutation**) com cache configurado.
 - Estados de loading visíveis em todas as telas que consomem API.
 - Timeout de requisição configurado para **60 segundos**.
 - Tratamento de erro de timeout/falha de rede com feedback ao usuário.

- **Prioridade:** 3
- **Esforço:** 5 pontos
- **Dependências técnicas:** PBI 4.3 (API JSON disponível); PBI 5.1.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T5.2.1	Configuração do client HTTP com timeout de 60s	1	PBI 4.3
T5.2.2	Implementação de queries/mutations (TanStack Query)	2	T5.2.1
T5.2.3	Estados de loading e tratamento de erro nas telas	2	T5.2.2

PBI 5.3 — Autenticação Externa e Desacoplada (Firebase)

- **Descrição:** Configurar autenticação via Firebase Authentication, integrada ao fluxo de login do app, mantendo a autenticação desacoplada da lógica de negócio.
- **História de Usuário:** Como tutor de pet, quero fazer login de forma simples e segura usando meu e-mail, para que eu possa acessar o cuidado contínuo do meu pet sem me preocupar com a segurança da minha conta.
- **Critérios de Aceite:**
 - Login/cadastro via Firebase Authentication funcional (e-mail/senha, no mínimo).
 - Token do Firebase validado/utilizado nas chamadas subsequentes à API.
 - Autenticação desacoplada da lógica de negócio (camada de serviço isolada).
 - Logout e persistência de sessão funcionando corretamente.
- **Prioridade:** 2
- **Esforço:** 5 pontos
- **Dependências técnicas:** Projeto Firebase configurado (Squad Cloud); PBI 5.1.

Tasks (Sprint 3):

Task	Descrição	Esforço	Dependências
T5.3.1	Configuração do projeto Firebase e SDK no app	1	—
T5.3.2	Login/cadastro (Firebase Auth)	2	T5.3.1
T5.3.3	Persistência de sessão e logout	1	T5.3.2
T5.3.4	Testes de fluxo de autenticação	1	T5.3.2, T5.3.3

FEATURE 6 — Integração Contínua e Operação (planejada para Sprint 4)

História de Usuário: Como time de engenharia, preciso que os serviços já construídos na Sprint 3 estejam integrados, testados de ponta a ponta e monitorados de forma centralizada, para que o KuraVet opere de forma confiável antes de ir para homologação.

PBI 6.1 — Pipeline CI/CD (Azure DevOps)

- **Descrição:** Configurar pipelines de build e deploy automatizado para os componentes desenvolvidos na Sprint 3 (banco Oracle, infraestrutura Azure, microsserviço .NET, monólito Java, app mobile).
- **História de Usuário:** Como desenvolvedor, quero que meu código seja testado e implantado automaticamente ao ser mesclado, para que eu não dependa de passos manuais propensos a erro para colocar algo em produção.
- **Critérios de Aceite:**
 - Pipeline de build/test/deploy configurado para cada componente principal.
 - Deploy automatizado dispara em merge na branch principal.
 - Pipeline falha e bloqueia deploy se os testes automatizados (PBI 3.2, entre outros) não passarem.
- **Prioridade:** 1
- **Esforço:** 13 pontos
- **Dependências técnicas:** Todos os PBIs da Sprint 3 concluídos e implantáveis.

PBI 6.2 — Testes End-to-End Integrados

- **Descrição:** Criar testes E2E cobrindo o fluxo completo do tutor (login mobile → consulta de dados via API Java → dados vindos do Oracle), validando a integração real entre todas as camadas.
- **História de Usuário:** Como QA do time, quero testes automatizados que simulem o fluxo real do tutor, para que eu detecte quebras de integração antes que cheguem à homologação.
- **Critérios de Aceite:**
 - Ao menos 1 fluxo E2E completo automatizado (login → navegação → consumo de dados reais).
 - Testes rodam no pipeline de CI/CD (PBI 6.1).
 - Relatório de execução com evidências anexado ao pipeline.
- **Prioridade:** 2
- **Esforço:** 13 pontos
- **Dependências técnicas:** PBI 6.1; App mobile e API Java implantados.

PBI 6.3 — Hardening de Compliance e Auditoria Cross-Sistema

- **Descrição:** Consolidar e revisar os pontos de auditoria/LGPD implementados isoladamente na Sprint 3 (trigger Oracle, autenticação, logs), garantindo cobertura cross-sistema.
- **História de Usuário:** Como responsável por compliance, quero uma visão consolidada de tudo que é auditado no KuraVet, para que eu consiga responder a uma auditoria externa com segurança.
- **Critérios de Aceite:**
 - Mapa de dados sensíveis e pontos de auditoria documentado.
 - Revisão de gaps de rastreabilidade entre camadas (Oracle, Java, mobile).
 - Checklist de compliance LGPD aplicado e assinado pelo responsável técnico.
- **Prioridade:** 2
- **Esforço:** 8 pontos
- **Dependências técnicas:** PBI 1.3 (trigger de auditoria) concluído.

PBI 6.4 — Observabilidade Centralizada (Dashboards)

- **Descrição:** Consolidar métricas e logs do microserviço .NET (PBI 3.1) e demais componentes em um dashboard único, dando visibilidade cross-squad da saúde do sistema.
- **História de Usuário:** Como líder técnico, quero um painel único com a saúde de todos os serviços, para que eu não precise checar cada componente separadamente para saber se está tudo funcionando.
- **Critérios de Aceite:**
 - Dashboard consolidado exibindo métricas de pelo menos 2 componentes.
 - Alertas configurados para indisponibilidade ou erro acima do limite.
- **Prioridade:** 2
- **Esforço:** 8 pontos
- **Dependências técnicas:** PBI 3.1; PBI 6.1.

PBI 6.5 — Notificações Push (Vacina/Consulta)

- **Descrição:** Novo incremento de escopo: notificar o tutor via push (Firebase Cloud Messaging) quando uma vacina ou consulta estiver próxima do vencimento, usando a regra de negócio já criada em **FN_CALC_STATUS_CUIDADO**.
- **História de Usuário:** Como tutor de pet, quero ser avisado automaticamente quando uma vacina ou consulta estiver perto do vencimento, para que eu não esqueça de cuidar da saúde do meu pet.
- **Critérios de Aceite:**
 - Push notification disparada quando o status do pet mudar para **ATRASADO** ou **CRITICO**.
 - Tutor consegue desativar notificações nas configurações do app.
 - Teste comprova o disparo correto em cenário de vencimento simulado.
- **Prioridade:** 3
- **Esforço:** 13 pontos
- **Dependências técnicas:** PBI 5.3 (Firebase); PBI 4.3 (API); PBI 1.2 (**FN_CALC_STATUS_CUIDADO**).

3. Release Plan (Roadmap de Entregas)

Visão geral por Sprint

Sprint	Foco	Esforço (pts)	Status
Sprint 1	Setup de ambientes, repositórios e pipelines base	—	Concluída
Sprint 2	Modelagem ER, wireframes de telas, PoCs de conectividade Azure/Firebase	—	Concluída
Sprint 3	Fundação técnica multi-camada (este backlog, em curso)	91	Em andamento
Sprint 4	Integração cross-squad, CI/CD, testes E2E, hardening LGPD, notificações	55	Planejada
Total do backlog planejado		146	

Mapeamento item a item (todo backlog planejado até aqui)

Sprint	PBI	Feature	Esforço (pts)
3	PBI 1.1 — Modelagem e Script Core do BD	Camada de Dados (Oracle)	8
3	PBI 1.2 — Procedures e Functions de Negócio	Camada de Dados (Oracle)	13
3	PBI 1.3 — Trigger de Auditoria	Camada de Dados (Oracle)	5
3	PBI 2.1 — Provisionamento via Azure CLI	Infraestrutura Azure	8
3	PBI 2.2 — CRUD Funcional na Nuvem	Infraestrutura Azure	8
3	PBI 3.1 — Health Check, Logs e Métricas	Microserviço .NET	8
3	PBI 3.2 — Testes Automatizados .NET	Microserviço .NET	5
3	PBI 4.1 — Views Thymeleaf e Flyway	Monólito Java	8
3	PBI 4.2 — Segurança por Perfis	Monólito Java	5
3	PBI 4.3 — API JSON Paralela para Mobile	Monólito Java	5
3	PBI 5.1 — Navegação e Telas Protegidas	App Mobile	8
3	PBI 5.2 — Integração de API (TanStack Query)	App Mobile	5
3	PBI 5.3 — Autenticação Firebase	App Mobile	5
3	Subtotal Sprint 3		91
4	PBI 6.1 — Pipeline CI/CD	Integração Contínua e Operação	13
4	PBI 6.2 — Testes E2E Integrados	Integração Contínua e Operação	13
4	PBI 6.3 — Hardening de Compliance Cross-Sistema	Integração Contínua e Operação	8
4	PBI 6.4 — Observabilidade Centralizada	Integração Contínua e Operação	8
4	PBI 6.5 — Notificações Push	Integração Contínua e Operação	13
4	Subtotal Sprint 4		55

Balanceamento entre Sprints: 91 → 55 pontos. A curva é decrescente de propósito: a Sprint 3 carrega mais peso por construir a fundação em paralelo nas 5 frentes técnicas (squads simultâneas); a Sprint 4 tem menos paralelismo possível (o trabalho passa a ser mais sequencial/integrador), então a capacidade foi reduzida para manter a carga sustentável por squad — evitando picos de esforço que gerem dívida técnica.

Balanceamento de esforço por squad na Sprint 3

Squad	PBIs	Esforço (pts)	% da Sprint
Dados (Oracle)	1.1, 1.2, 1.3	26	28,6%
Cloud (Azure)	2.1, 2.2	16	17,6%
Backend .NET	3.1, 3.2	13	14,3%
Backend Java	4.1, 4.2, 4.3	18	19,8%
Mobile (React Native)	5.1, 5.2, 5.3	18	19,8%
Total	13 PBIs / 51 Tasks	91	100%

Observação de capacidade: 91 pontos distribuídos entre 5 squads paralelas resulta em 13–26 pontos por squad — carga saudável para uma sprint de 2 semanas por squad especializada. Caso a equipe seja única (sem paralelismo real entre squads), recomenda-se dividir esta Sprint 3 em duas iterações (3a e 3b), priorizando PBIs de Prioridade 1 (1.1, 1.2, 2.1) na primeira metade.

Ordem de dependências críticas (caminho crítico)

PBI 1.1 → PBI 1.2 / PBI 1.3 (Dados) · PBI 2.1 → PBI 2.2 (Cloud) · PBI 4.1 → PBI 4.2 → PBI 4.3 (Java) · PBI 5.3 → PBI 5.1 → PBI 5.2 (Mobile, autenticação antes de navegação protegida e antes do consumo de API)

4. Definition of Done (Compliance e Qualidade)

Um item só é considerado "Done" quando:

- ☐ Critérios de aceite validados e evidenciados (print/log/execução).
- ☐ Código revisado por par (Pull Request aprovado).
- ☐ Testes automatizados (quando aplicável) executando com sucesso no pipeline.
- ☐ Sem segredos/credenciais em texto plano no código ou nos scripts.
- ☐ Alterações em dados sensíveis rastreáveis via auditoria (LGPD).
- ☐ Documentação técnica mínima atualizada (README/Swagger/comentários de script).
- ☐ Item validado pelo Product Owner ou responsável técnico da squad.

5. Notas para importação no Azure Boards

O arquivo [KuraVet_Sprint3_AzureBoards_Import.csv](#) traz as 70 linhas (1 Épico, 5 Features, 13 PBIs, 51 Tasks) da **Sprint 3** já formatadas para importação via **Boards → Work Items → Import Work Items** — e já foram efetivamente importadas e vinculadas no projeto KuraVet no Azure DevOps. As 5 PBIs da Feature 6 (Sprint 4) ainda não foram criadas no Azure Boards — elas fazem parte do planejamento de roadmap deste documento e devem ser importadas/criadas quando a Sprint 4 for iniciada. Como o CSV nativo do Azure Boards não cria relações Pai/Filho automaticamente, siga:

1. Importe o CSV normalmente (cria os work items sem hierarquia).
2. Abra a view de **Backlog** ou **Queries** → **New Query**, agrupe por **Parent Title (referência)** (coluna auxiliar do CSV, não é um campo nativo).
3. Use **Boards** → **Backlogs** (arrastar item para dentro do Épico/Feature/PBI) ou a função **Bulk Edit** para setar o campo **Parent** de cada item, usando a coluna auxiliar como guia.
4. Alternativa mais rápida para times com CLI: usar **az boards work-item create --parent** via um script que itere o CSV — posso gerar esse script se for útil.